
Modding de Armas en Terraria

*Investigación exhaustiva de proyectos
de GitHub y documentación técnica*

- **APIS DE tModLoader**
- **CATÁLOGO DE REPOS GITHUB**
- **PATRONES DE CÓDIGO C#**
- **CLASES DE DAÑO**
- **DISTRIBUCIÓN STEAM WORKSHOP**

PROPÓSITO

Documento de contexto para alimentar a un modelo de IA.

Audiencia: IA / desarrollador de mods de Terraria.

Idioma: Español · Profundidad: Enciclopédica

Cobertura: tModLoader 1.4.4 · Todas las clases de armas

Índice

1. Resumen Ejecutivo	6
2. Panorama del Ecosistema tModLoader	7
2.1 Qué es tModLoader	7
2.2 Distribución: Steam Workshop vs Mod Browser	7
2.3 Licencia y gobernanza	7
3. Setup del Entorno de Desarrollo	8
3.1 Requisitos previos	8
3.2 Activar Developer Mode en tModLoader	8
3.3 Instalación de Visual Studio y .NET SDK	8
3.4 Plantilla de proyecto tModLoader	8
3.5 Depuración (debugging)	9
4. Arquitectura de un Mod	9
4.1 Estructura de carpetas	9
4.2 build.txt — Metadatos de compilación	10
4.3 Autoload: cómo tModLoader descubre tus clases	10
4.4 La clase Mod principal	10
4.5 Convenciones de naming	11
5. API de ModItem - Hooks Fundamentales	11
5.1 SetStaticDefaults vs SetDefaults	11
5.2 Hooks principales de ModItem	12
5.3 Ejemplo completo: espada básica	12
6. API de ModProjectile - Projectiles Personalizados	13
6.1 Por qué ModProjectile es crítico	13
6.2 Hooks de ModProjectile	14
6.3 Sistema AIStyle	14
6.4 AI custom: timers y movimiento	15
7. DamageClass - Sistema de Clases de Daño	16
7.1 Por qué DamageClass importa	16
7.2 Clases vanilla	16
7.3 Crear DamageClass custom	16

7.4 Modificadores: cómo escala el daño	17
8. Creación de Armas Melee	17
8.1 Espadas simples	17
8.2 Espadas que disparan proyectiles	18
8.3 Yoyos	18
8.4 Flails	19
8.5 Boomerangs	19
9. Creación de Armas Ranged	19
9.1 Sistema de munición	19
9.2 Arco básico	20
9.3 Gun básico	20
9.4 Munición custom	20
9.5 Patrón Daedalus Stormbow	21
10. Creación de Armas Magic	21
10.1 Configuración básica	21
10.2 Patrones de disparo	22
10.3 Sistema de carga (charge-up)	22
11. Creación de Armas Summon	23
11.1 Tipos de armas summon	23
11.2 Minions	23
11.3 Sentries	24
11.4 Whips	25
12. Creación de Armas Throwing/Rogue	25
12.1 Estado de la clase Throwing	25
12.2 Item throwing básico	26
12.3 Clase Rogue de Calamity	26
13. Sistema de Accesorios	26
13.1 ModItem como accesorio	26
13.2 ModPlayer para efectos custom	27
14. Sistema de Recetas	27
14.1 Recipe.Create API	27

14.2 Recipe Groups	27
14.3 Cross-mod en recetas	28
15. Catálogo de Repositorios de GitHub	28
15.1 tModLoader (framework base)	28
15.2 CalamityModPublic (mirror del mod más grande)	29
15.3 ThoriumMod	29
15.4 WeaponEnchantments	30
15.5 AmmoboxPlus	30
15.6 TerraScape	30
15.7 ArkknightsMod	31
15.8 Persona5Cosplay	31
15.9 CalamityEntropy (addon de Calamity)	31
15.10 AlchemistNPC	32
15.11 ie53-Terraria-All-Items-Mod	32
15.12 tModLoader ExampleMod	32
15.13 Otros repos menores y recursos	33
16. Análisis Comparativo de Arquitecturas	33
16.1 Organización por clase de daño (Calamity)	33
16.2 Organización por tipo de contenido (ExampleMod)	33
16.3 Organización por tier de progresión (Thorium)	33
16.4 Buenas prácticas transversales	33
16.5 Anti-patrones comunes	34
17. Distribución y Publicación	34
17.1 Pipeline de build y test	34
17.2 Publicar en Steam Workshop	34
17.3 Mod References en build.txt	34
17.4 GitHub Releases como mirror	35
17.5 Versionado semántico	35
18. Pitfalls Comunes y Debugging	35
18.1 NullReferenceException en SetDefaults	35
18.2 Projectile no aparece al disparar	35
18.3 Sprite no carga / aparece como cuadrado rojo	35

18.4 Daño no aplica correctamente	36
18.5 Multiplayer desync	36
18.6 OutOfMemoryException al cargar	36
18.7 Build errors comunes	36
18.8 Cómo leer los logs	37
19. Patrones de Cross-Mod Compatibility	37
19.1 Mod Calls (sistema de mensajería)	37
19.2 TryFind para items y proyectiles	37
19.3 Weak references en build.txt	37
19.4 Compat con mods utilitarios	38
19.5 Patrón de Mod addon	38
20. Snippets de Código Reales	38
20.1 Espada básica con tooltip y receta	39
20.2 Magic weapon con spread de 5 proyectiles	39
20.3 Accesorio que da vida y defensa	40
20.4 ModSystem con hooks globales	40
20.5 GlobalItem que modifica todos los items	41
20.6 ModPlayer con estado custom	41
21. Patrones de Generación para IA	42
21.1 Reglas obligatorias al generar código	42
21.2 Plantilla de prompt sugerida	42
21.3 Checklist de validación post-generación	42
21.4 Patrones anti-patrones que la IA debe evitar	43
22. Recursos y Referencias Adicionales	43
22.1 Documentación oficial	43
22.2 Comunidad	44
22.3 Herramientas y utilidades	44
22.4 Tutoriales recomendados	45
23. Conclusión y Recomendaciones	45
23.1 Próximos pasos sugeridos	45
23.2 Checklist final para cualquier mod de armas	46

1. Resumen Ejecutivo

Este documento compila una investigación exhaustiva sobre el ecosistema de mods de armas para Terraria, con énfasis en proyectos de GitHub públicos y en la API oficial de tModLoader. Su propósito es servir como documento de contexto para que un modelo de IA pueda responder preguntas técnicas, generar código funcional, comparar arquitecturas de mods y proponer extensiones sobre mods existentes. La cobertura abarca desde la instalación del entorno de desarrollo hasta la publicación en Steam Workshop, pasando por todos los hooks de la API de ModItem y ModProjectile, el sistema de clases de daño, los patrones de cross-mod compatibility y los errores más comunes que enfrenta un desarrollador novel.

La metodología de investigación ha combinado búsquedas dirigidas en GitHub (topics: tmodloader-mod, calamitymod), lectura de la documentación oficial en docs.tmodloader.net, revisión de tutoriales activos en forums.terraria.org, y análisis del código fuente del ExampleMod incluido en el repositorio principal de tModLoader. Cada repositorio mencionado en el catálogo ha sido verificado en cuanto a su URL, autor, características distintivas y relevancia para el aprendizaje. Los snippets de código están redactados en C# y siguen las convenciones de tModLoader 1.4.4 (versión estable actual).

El documento se estructura en 23 capítulos que cubren: el ecosistema general, setup del entorno, arquitectura de un mod, las APIs centrales (ModItem, ModProjectile, DamageClass, GlobalItem, ModPlayer, ModSystem), implementación específica para cada clase de arma (melee, ranged, magic, summon, throwing/rogue), accesorios, sistema de recetas, catálogo detallado de 13 repositorios de referencia, análisis comparativo de arquitecturas, distribución y publicación, pitfalls comunes, cross-mod compatibility, snippets de código listos para usar, y patrones de generación orientados a IA. El lector encontrará, al final, un compendio de URLs críticas y una sección de conclusiones con recomendaciones operativas.

La audiencia objetivo es mixta: una IA puede usar este documento como base para generar código de mods, para explicar conceptos a usuarios humanos, o para extender mods existentes como Calamity o Thorium. Por ello, el nivel de detalle técnico es alto (incluye nombres exactos de clases, hooks, namespaces y archivos de configuración), pero el tono es accesible y los conceptos se introducen gradualmente. Donde es pertinente, se incluyen comparaciones entre enfoques alternativos (por ejemplo, ModProjectile.AI() custom versus AIStyle vanilla) con sus ventajas y desventajas.

2. Panorama del Ecosistema tModLoader

2.1 Qué es tModLoader

tModLoader (abreviado TML) es un mod open-source que actúa como capa de modificación y expansión sobre el juego Terraria, desarrollado por Re-Logic. Su función esencial es proveer una API en C#/.NET que permite a cualquier desarrollador crear contenido nuevo (armas, NPCs, bloques, jefes, biomas, mecánicas) sin tener que modificar el ejecutable original del juego. TML se distribuye gratuitamente como DLC de Terraria en Steam, lo que significa que para usarlo solo se necesita tener Terraria instalado en la biblioteca de Steam y descargar tModLoader por separado. La organización GitHub oficial es github.com/tModLoader, y el repositorio principal mantiene vías paralelas para la versión 1.4-stable y la preview 1.4-preview.

El historial del proyecto incluye una transición desde tAPI (la API legacy usada en la época de Terraria 1.2), pasando por tModLoader 1.3 (que acompañó al juego durante años), hasta la actual versión 1.4 que corresponde a la última generación de Terraria (1.4.4 Labor of Love). Cada salto de versión introdujo cambios sustanciales en la API: en 1.4 se reescribió por completo el sistema de DamageClass, se eliminó ModRecipe en favor del sistema Recipe.Create(), y se migró de .NET Framework 4.x a .NET 6/8. La comunidad mantiene viva la rama 1.3-Legacy para mods antiguos que nunca se portaron, pero todo desarrollo nuevo se hace sobre 1.4.4.

2.2 Distribución: Steam Workshop vs Mod Browser

A partir de la versión Steam de tModLoader, el canal principal de distribución de mods es el Steam Workshop. Esto reemplazó al antiguo Mod Browser in-game, que era un servidor mirror mantenido por la comunidad. El Steam Workshop ofrece ventajas significativas: suscripción con un clic, actualizaciones automáticas, estadísticas de descargas, y soporte para capturas de pantalla y videos. El Mod Browser sigue existiendo como fallback para versiones legacy y para usuarios de GOG (que no tienen Steam), pero la inmensa mayoría del tráfico actual pasa por Workshop. Para un desarrollador, publicar en Workshop es obligatorio si quiere llegar a una audiencia amplia.

2.3 Licencia y gobernanza

tModLoader se publica bajo licencia MIT, lo que permite uso, modificación y redistribución libres, incluso con fines comerciales (sujeto a mantener el aviso de copyright). La organización GitHub tModLoader es mantenida por un equipo de colaboradores voluntarios coordinados por los desarrolladores originales (blushiemagic, Scal, Solxan, Snorkleboy y otros). Las decisiones de diseño se discuten abiertamente en GitHub Issues y en el servidor Discord oficial. Los mods individuales, sin embargo, retienen su propia licencia: Calamity usa una licencia restrictiva (código público pero assets sujetos a copyright), mientras que ExampleMod y muchos mods pequeños son MIT. Es responsabilidad del modder verificar la licencia antes de reutilizar código o assets de otro mod.

3. Setup del Entorno de Desarrollo

3.1 Requisitos previos

Antes de escribir una sola línea de código, es necesario preparar el entorno con los componentes correctos. Los requisitos mínimos son: sistema operativo Windows 10/11, macOS 11+ o Linux moderno; Terraria instalado (comprado en Steam o GOG); tModLoader instalado como DLC gratuito de Steam; Visual Studio Community 2022 (gratuito, versión Windows) o Visual Studio Code / Rider para macOS/Linux; y el SDK de .NET 6.0 (o superior, compatible con .NET 8). La combinación de Visual Studio + .NET SDK + tModLoader es la más documentada y la que menos fricciones presenta, especialmente para principiantes.

3.2 Activar Developer Mode en tModLoader

Una vez instalado tModLoader desde Steam, hay que iniciar el juego una primera vez para que se generen las carpetas de configuración. Luego, en el menú principal, se navega a Settings -> Developer Mode (o, en versiones recientes, Workshop -> Manage Mods -> Developer Settings) y se activa la opción "Enable Developer Mode". Esto desbloquea el botón "Mod Sources" en el menú principal, que abre la carpeta donde se colocan los proyectos de mod. Esta carpeta está típicamente en:

```
Windows: %USERPROFILE%\Documents\My Games\Terraria\tModLoader\ModSources\  
macOS:   ~/Library/Application Support/Terraria/tModLoader/ModSources/  
Linux:   ~/.local/share/Terraria/tModLoader/ModSources/
```

Dentro de tModLoader, en el menú Developer Settings, también es recomendable activar "Automatically reload mods when sources change" y "Pause game when window loses focus" para acelerar el ciclo de edición-prueba.

3.3 Instalación de Visual Studio y .NET SDK

Para Windows, se descarga Visual Studio Community 2022 desde visualstudio.microsoft.com. Durante la instalación, en el instalador, se debe seleccionar obligatoriamente el workload ".NET desktop development", que incluye el compilador de C#, las plantillas para proyectos WinForms/WPF/Console, y el soporte para depuración. Para macOS/Linux, se instala Visual Studio Code con la extensión C# de Microsoft, o bien JetBrains Rider. Adicionalmente, se descarga e instala el .NET SDK 6.0 desde dotnet.microsoft.com. La forma de verificar que todo está correcto es abrir una terminal y ejecutar `dotnet --version`, que debe devolver un número igual o superior a 6.0.x.

3.4 Plantilla de proyecto tModLoader

tModLoader incluye un Mod Generator in-game accesible desde Workshop -> Develop Mods -> Create Mod. Al crear un mod nuevo, se solicita un nombre interno (sin espacios, solo alfanumérico, ej: "ExampleMod"), un display name, y un autor. El generador crea automáticamente la estructura de carpetas mínima con build.txt, description.txt, icon.png y un proyecto .csproj listo para abrir en Visual Studio. Alternativamente, se puede instalar la plantilla dotnet desde la línea de comandos:


```
dotnet new --install tModLoader.Templates
dotnet new tmodloader-mod -n MiModDeArmas
cd MiModDeArmas
code . # o "start ." en Windows para abrir el explorador
```

Ambos enfoques producen el mismo resultado: una carpeta de mod con un .csproj configurado para referenciar los assemblies de tModLoader (que residen en la carpeta de instalación del juego). Para que Visual Studio pueda resolver estas referencias, es necesario que tModLoader esté instalado vía Steam; el .csproj usa MSBuild properties que localizan automáticamente la carpeta de Steam.

3.5 Depuración (debugging)

Existen dos formas principales de depurar un mod. La primera es construir y ejecutar desde dentro del juego: en Workshop -> Develop Mods, se selecciona el mod y se pulsa "Build + Reload". Esto recompila el C# y recarga el mod en caliente; cualquier excepción se mostrará en la consola de tModLoader y se registrará en el archivo de log. La segunda, más avanzada, es usar Visual Studio attach-to-process: se inicia tModLoader normalmente, luego en Visual Studio se usa Debug -> Attach to Process y se selecciona tModLoader.exe. A partir de ahí, se pueden poner breakpoints en el código del mod y la ejecución se detendrá al pasar por ellos. Esta segunda forma es esencial para depurar bugs sutiles en hooks como AI() o ModifyHitNPC() donde los Logs no dan suficiente contexto.

Los logs de tModLoader se encuentran en Documents/My Games/Terraria/tModLoader/Logs/ (Windows). El archivo principal es server.log para mods en servidor dedicado, y client.log para cliente. En cada sesión se crea un archivo nuevo con timestamp. Es buena práctica abrir el log más reciente después de cada build para verificar que no hay warnings de compilación ni errores de autoloading.

4. Arquitectura de un Mod

4.1 Estructura de carpetas

Todo mod de tModLoader se organiza en una carpeta raíz cuyo nombre coincide con el nombre interno del mod (el que aparece en build.txt). Dentro de esa carpeta, los archivos mínimos son build.txt (metadatos de compilación), description.txt (texto mostrado en el menú de mods in-game), e icon.png (256x256 píxeles, ícono del mod). El código C# se coloca típicamente en subcarpetas bajo una raíz que coincide con el namespace principal. La convención de ExampleMod y de la mayoría de mods grandes es:

```
MiMod/
├── build.txt
├── description.txt
├── icon.png
├── MiMod.csproj
├── MiMod.cs # clase Mod principal (override Load/Unload)
├── Content/
│   ├── Items/
│   │   ├── Weapons/
│   │   │   ├── ExampleSword.cs
│   │   │   ├── ExampleBow.cs
│   │   │   └── ExampleGun.cs
│   │   └── Accessories/
│   └── Materials/
└── Projectiles/
```

```

|   |   | Melee/
|   |   | Ranged/
|   |   | Magic/
|   |   |
|   |   | NPCs/
|   |   | Tiles/
|   |   | Buffs/
|   |   | Systems/
|   |   |
|   |   | # ModSystem classes

```

4.2 build.txt — Metadatos de compilación

El archivo build.txt contiene pares clave=valor que tModLoader lee al compilar el mod. Los campos principales son:

```

author = Tu Nombre
version = 1.0.0
displayName = Mi Mod de Armas
homepage = https://github.com/usuario/mi-mod-de-armas
modReferences = CalamityMod @ 2.0.4.5
dllReferences = Newtonsoft.Json
weakReferences = ThoriumMod
sortAfter = CalamityMod
sortBefore = CheatSheet
side = Both

```

El campo `modReferences` lista mods de los que tu mod depende obligatoriamente (TML no cargará tu mod si los mods referenciados no están presentes). `weakReferences` es similar pero opcional: tu mod carga aunque el mod referenciado no esté, pero si está, se activa contenido cross-mod. `dllReferences` permite referenciar DLLs externas (NuGet packages o librerías custom). `side` indica en qué lados se ejecuta el mod: Both (cliente y servidor), Client (solo cliente, ej: mods visuales), Server (solo servidor), o NoSync (cliente sin afectar sincronización). Para mods de armas, lo normal es `side = Both`.

4.3 Autoload: cómo tModLoader descubre tus clases

tModLoader usa reflection para detectar automáticamente las clases que heredan de ModItem, ModProjectile, ModNPC, ModTile, ModBuff, ModSystem, ModPlayer, GlobalItem, GlobalProjectile, etc. Este mecanismo se llama "autoload" y es la forma idiomática de registrar contenido. Para que autoload funcione, la clase debe ser public, no abstract, tener un constructor sin parámetros, y estar dentro del namespace correcto (que coincide con el nombre interno del mod). Por convención, el path de la textura PNG se infiere del namespace + nombre de la clase: si tu clase es MiMod.Content.Items.Weapons.ExampleSword, tModLoader buscará un archivo en MiMod/Content/Items/Weapons/ExampleSword.png. Si quieres usar un path distinto, puedes overridear la propiedad Texture en la clase, o usar el atributo [Texture("MiMod/Assets/Items/MySword")].

4.4 La clase Mod principal

Cada mod tiene opcionalmente una clase que hereda de Terraria.ModLoader.Mod. Esta clase se usa para hooks globales del mod: Load() (se ejecuta al cargar el mod), Unload() (al descargar, para limpieza), PostSetupContent() (después de que todo el contenido está cargado, útil para cross-mod), y Call() (para cross-mod messaging). La clase se decora con [Autoload] para que tModLoader la instancie automáticamente. Ejemplo mínimo:

```

using Terraria.ModLoader;

namespace MiModDeArmas
{
    [Autoload]
    public class MiModDeArmas : Mod
    {
        public override void Load()
        {
            // Inicialización: registrar instancias, leer config, etc.
        }

        public override void Unload()
        {
            // Limpieza: liberar recursos, desuscribir eventos
        }
    }
}

```

4.5 Convenciones de naming

Los mods grandes siguen convenciones estrictas de naming para mantener el código navegable. Calamity usa prefijos por categoría: "BladecrestOathsword" (melee sword), "MegafleetShell" (ranged ammo), "SanguineFlare" (magic), "AnechoicPlating" (accessory). ExampleMod prefija todo con "Example" para evitar colisiones. Thorium usa nombres temáticos (Terrarian, Berserker, etc.) sin prefijo fijo. La regla práctica es: elige una convención y mantén la consistencia. Evita nombres genéricos como "Sword" o "Bow" porque colisionarán con otros mods. Para nombres internos (campos, métodos), usa PascalCase para propiedades públicas y camelCase para privados, siguiendo las convenciones de C#.

5. API de ModItem - Hooks Fundamentales

5.1 SetStaticDefaults vs SetDefaults

Es la distinción más importante y la que más confunde a los principiantes. `SetStaticDefaults()` se ejecuta una sola vez al cargar el mod y se usa para propiedades que afectan al tipo de ítem en su totalidad, no a una instancia concreta. Aquí se definen `DisplayName.SetDefault("Nombre")`, `Tooltip.SetDefault("Descripción")`, y los sets de `ItemID.Sets` (como `ItemID.Sets.ItemsThatCountAsBombsForDemolitionistToSpawn`). `SetDefaults()` se ejecuta cada vez que se crea una instancia del ítem (por ejemplo, cada vez que se genera en el inventario de un jugador) y se usa para stats: `damage`, `width`, `height`, `useStyle`, `useTime`, `useAnimation`, `knockBack`, `value`, `rare`, `DamageType`, `UseSound`, `shoot`, `shootSpeed`, `useAmmo`, `autoReuse`, etc. La regla mnemotécnica: si la propiedad describe "qué es el ítem" (nombre, tooltip, comportamiento del tipo), va en `SetStaticDefaults`; si describe "qué hace el ítem" (daño, velocidad, `sprite size`), va en `SetDefaults`.

5.2 Hooks principales de ModItem

Los hooks más usados al crear armas son los siguientes. Cada uno se invoca en un momento específico del ciclo de uso del item y permite modificar el comportamiento:

Hook	Cuándo se invoca	Uso típico
SetStaticDefaults()	Una vez al cargar el mod	DisplayName, Tooltip, ItemID.Sets
SetDefaults()	Al crear instancia del item	damage, useStyle, useTime, DamageType
CanUseItem(Player)	Al intentar usar el item	Verificar condiciones (mana, cooldown, etc.)
UseItem(Player)	Tras activarse el item	Aplicar efectos custom, spawn de proyectiles
HoldItem(Player)	Cada frame mientras se sostiene	Efectos pasivos (luz, partículas, buffs)
UseAnimation(Player)	Cada frame durante la animación	Sonidos progresivos, partículas
HoldStyle(Player)	Cada frame mientras se sostiene	Posición custom del item en mano
UseStyle(Player)	Cada frame durante el uso	Movimiento custom del item al usar
Shoot(Player, pos, vel, type, dmg, kb)	Tras UseItem, antes de spawn	Spawnear proyectiles custom
OnHitNPC(Player, NPC, dmg, kb, crit)	Al golpear un NPC	Aplicar debuffs, vida drain, knockback extra
ModifyHitNPC(Player, NPC, ref dmg, ref kb, ref crit)	Antes de golpear NPC	Modificar dmg/kb/crit
CanHitNPC(Player, NPC)	Verificar si puede golpear	Devolver false para bloquear golpe
AddRecipes()	Al cargar el mod	Registrar recetas de crafteo

Cada hook devuelve void por defecto, excepto CanUseItem (bool), Shoot (bool, devolver false para cancelar el spawn del proyectil vanilla), CanHitNPC (bool?), y ModifyHitNPC (usa parámetros ref). Es importante llamar a base.Method() en los overrides cuando se quiere preservar el comportamiento vanilla (raramente necesario en ModItem, más común en GlobalItem).

5.3 Ejemplo completo: espada básica

A continuación se muestra el código mínimo de una espada melee que hace 25 de daño, tiene autoswing, y aplica el debuff OnFire al golpear. Este ejemplo cubre los hooks más comunes y sirve como plantilla para crear cualquier arma melee:

```
using Terraria;
using Terraria.ID;
using Terraria.ModLoader;
using Terraria.DataStructures;

namespace MiMod.Content.Items.Weapons
{
    public class EspadaLlameante : ModItem
```

```

{
    public override void SetStaticDefaults()
    {
        DisplayName.SetDefault("Espada Llaveante");
        Tooltip.SetDefault("\\c/FF8800:Inflige OnFire al golpear\\c/.");
    }

    public override void SetDefaults()
    {
        Item.damage = 25;
        Item.DamageType = DamageClass.Melee;
        Item.width = 40;
        Item.height = 40;
        Item.useTime = 20;
        Item.useAnimation = 20;
        Item.useStyle = ItemUseStyleID.Swing;
        Item.knockBack = 5f;
        Item.value = Item.buyPrice(silver: 50);
        Item.rare = ItemRarityID.Blue;
        Item.UseSound = SoundID.Item1;
        Item.autoReuse = true;
    }

    public override void OnHitNPC(Player player, NPC target,
                                   int damage, float knockBack, bool crit)
    {
        target.AddBuff(BuffID.OnFire, 180); // 3 segundos de OnFire
    }

    public override void AddRecipes()
    {
        Recipe.Create(ModContent.ItemType<EspadaLlaveante>())
            .AddIngredient(ItemID.WoodenSword, 1)
            .AddIngredient(ItemID.Torch, 10)
            .AddTile(TileID.WorkBenches)
            .Register();
    }
}

```

► Observa que `Item.DamageType = DamageClass.Melee` es **OBLIGATORIO** en `SetDefaults`. Sin esta línea, el ítem cae en `DamageClass.Default` y no recibe bonificadores de daño melee del jugador.

6. API de ModProjectile - Projectiles Personalizados

6.1 Por qué ModProjectile es crítico

En Terraria, la mayoría de armas no dañan al enemigo directamente: generan un Projectile que es lo que realmente inflige daño. Esto incluye no solo arcos y guns (que disparan flechas/balas), sino también espadas que sueltan proyectiles tipo "slash" (como la Terra Blade), yoyos (que son proyectiles sostenidos), flails, boomerangs, whips, minions, sentries y todas las armas mágicas. Entender ModProjectile es por tanto esencial para cualquier weapon modder. La clase ModProjectile envuelve al Terraria.Projectile subyacente y expone hooks para customizar su comportamiento, especialmente su AI.

6.2 Hooks de ModProjectile

Hook	Cuándo	Uso
SetStaticDefaults()	Una vez al cargar	DisplayName, Main.projFrames, ProjectileID.Sets
SetDefaults()	Al spawnear	width, height, aiStyle, friendly, hostile, penetrate, timeLeft, light
AI()	Cada frame mientras activo	Lógica custom: movimiento, homing, timers
OnTileCollide(Vector2)	Al chocar con tile	Rebotar, destruir, spawn de partículas
OnHitNPC(NPC, dmg, kb, crit)	Al golpear NPC	Bufs, vida drain, spawn de subproyectiles
Kill(int, int)	Al morir/expires	Partículas, sonido, drop de items
FindFrame(int)	Cada frame	Animación de spritesheet
ModifyHitNPC(NPC, ref dmg, ref kb, ref crit, ref dmgType)	Antes de golpear	Modificar daño
GetAlpha(Color)	Cada frame	Transparencia / tint
PreDraw / PostDraw	Render	Dibujo custom

6.3 Sistema AIStyle

tModLoader expone los AI styles vanilla via ProjAIStyleID. En lugar de escribir AI custom desde cero, puedes asignar Projectile.aiStyle = ProjAIStyleID.Something en SetDefaults y tModLoader ejecutará la AI vanilla. Esto es útil para proyectiles simples (flechas, bolas de fuego) donde la AI vanilla es suficiente. Para personalizar parcialmente una AI vanilla, se usa Projectile.aiStyle + AIType (un ProjectileID cuyo comportamiento se mimetiza) y luego se overridea AI() agregando lógica adicional. Algunos AI styles comunes:

AIStyle	ID	Comportamiento
Sword	1	Espadas que se desvanecen
Boomerang	3	Retorna al jugador
Flail	15	Cadena + bola
Arrow / Bullet	2	Proyectil recto con gravedad
Magic bullet	9	Homing débil
Yoyo	99 (custom)	Sostenido por el jugador
Minion	default	Sigue al jugador, ataca enemigos

6.4 AI custom: timers y movimiento

El array `Projectile.ai[]` es el mecanismo estándar para almacenar estado entre frames. Tiene 2 floats (`ai[0]` y `ai[1]`) y, en casos especiales, se puede usar `Projectile.localAI[0..2]` para estado client-only. Patrón típico: `ai[0]` como timer (incrementa cada frame), `ai[1]` como "estado" (0 = buscando, 1 = atacando). Ejemplo de proyectil homing que busca al NPC más cercano después de 30 frames:

```
public override void AI()
{
    Projectile.ai[0]++; // timer

    // Buscar target después de 30 frames
    if (Projectile.ai[0] > 30 && Projectile.ai[1] == 0)
    {
        NPC closest = null;
        float minDist = 500f;
        foreach (NPC npc in Main.npc)
        {
            if (!npc.active || !npc.CanBeChasedBy(Projectile)) continue;
            float dist = Vector2.Distance(npc.Center, Projectile.Center);
            if (dist < minDist)
            {
                minDist = dist;
                closest = npc;
            }
        }
        if (closest != null)
        {
            Projectile.ai[1] = closest.whoAmI; // store target
        }
    }

    // Homing
    if (Projectile.ai[1] > 0)
    {
        NPC target = Main.npc[(int)Projectile.ai[1]];
        if (target.active)
        {
            Vector2 toTarget = target.Center - Projectile.Center;
            toTarget.Normalize();
            Projectile.velocity = Vector2.Lerp(
                Projectile.velocity, toTarget * 10f, 0.1f);
        }
    }

    // Rotación visual
    Projectile.rotation = Projectile.velocity.ToRotation() + MathHelper.PiOver2;
}
```

► Usar `Projectile.ai[]` para estado es **CRÍTICO** en multiplayer: estas variables se sincronizan automáticamente. **NO** uses campos estáticos o variables de instancia para estado, porque causarán desync entre cliente y servidor.

7. DamageClass - Sistema de Clases de Daño

7.1 Por qué DamageClass importa

En Terraria 1.3, las clases de daño eran simples strings o enums: "melee", "ranged", "magic", "summon", "thrown". Cada item tenía un campo item.melee, item.ranged, etc. (booleanos). Este diseño era limitado: no permitía clases custom, ni armas híbridas que escalaran con dos clases, ni modificar cómo se aplicaban los multiplicadores. En tModLoader 1.4, este sistema se reemplazó por DamageClass, una jerarquía de clases orientada a objetos que permite crear clases custom y definir exactamente cómo escalan los modificadores. Esta refactorización está documentada en el issue #1173 del repositorio tModLoader.

7.2 Clases vanilla

DamageClass	Uso	Ejemplo vanilla
Default	Sin clase definida	Items misceláneos, materiales
Melee	Cuerpo a cuerpo	Copper Shortsword, Terra Blade
Ranged	A distancia (usa ammo)	Wooden Bow, Minishark
Magic	Mágico (usa mana)	Water Bolt, Demon Scythe
Summon	Invocación	Slime Staff, Stardust Dragon Staff
SummonMeleeSpeed	Whips (afecta melee speed)	Leather Whip, Kaleidoscope
MagicSummonHybrid	Híbrido magic+summon	Forbidden Storm (set bonus)
Throwing	Arrojadizos (legacy, 1.3)	No usado en 1.4 vanilla, mantenido para mods

7.3 Crear DamageClass custom

Crear una clase de daño custom implica heredar de DamageClass y overridear varias propiedades y métodos. El ejemplo canónico es la Rogue class de Calamity, pero el patrón se aplica a cualquier clase nueva. Pasos:

```
using Terraria.ModLoader;

namespace MiMod.Content.DamageClasses
{
    public class RogueDamageClass : DamageClass
    {
        public override void SetStaticDefaults()
        {
            DisplayName.SetDefault("rogue");
        }

        // Cómo muestra el nombre en tooltips: "X rogue damage"
```



```

    public override string GetTooltip(Player player) => "rogue";

    // Permite que esta clase escale con bonus genéricos
    public override bool UseStandardCritCalculus => true;

    // Stats que esta clase hereda por defecto
    public override StatInheritanceData GetModifierInheritance(DamageClass
inheritor)
    {
        // Rogue hereda melee (armelee) y algo de ranged
        if (inheritor == DamageClass.Melee)
            return new StatInheritanceData(
                damageInheritance: 0.5f, critChanceInheritance: 0.5f,
                attackSpeedInheritance: 0.5f, armorPenInheritance: 0.5f,
                knockbackInheritance: 0.5f);
        return StatInheritanceData.Full;
    }

    public override bool ShowStatTooltip() => true;
}

```

Una vez creada la clase, se asigna a items con `Item.DamageType = ModContent.GetInstance<RogueDamageClass>()` en `SetDefaults`. Para crear accesorios que mejoren esta clase, se modifica `player.GetDamage<RogueDamageClass>()` en `UpdateAccessory`. El sistema es flexible pero tiene una curva de aprendizaje: revisar el source de Calamity es la mejor forma de entender patrones avanzados.

7.4 Modificadores: cómo escala el daño

Cuando un jugador golpea con un arma, el daño final se calcula multiplicando el daño base del item por varios modificadores: `player.GetDamage(DamageType)` (que incluye bonus por accesorios, armadura, buffs, y bonuses planos), `player.GetCritChance(DamageType)` (probabilidad de crítico), `player.GetAttackSpeed(DamageType)` (velocidad de uso, solo afecta melee y whips), `player.GetArmorPenetration(DamageType)` (penetración de armadura), y `player.GetKnockback(DamageType)`. Para una `DamageClass` custom, todos estos getters funcionan automáticamente si configuras correctamente `GetModifierInheritance`. Por ejemplo, una clase "Rogue" que herede 50% de melee hará que un accesorio +10% melee damage también dé +5% rogue damage.

8. Creación de Armas Melee

8.1 Espadas simples

La espada es el arquetipo más simple de arma melee: daño, knockback, useStyle Swing, autoswing opcional, y opcionalmente `OnHitNPC` para efectos custom. La espada básica mostrada en el capítulo 5 es el template canónico. Variaciones comunes: añadir `item.shoot = ProjectileID.X` para que la espada dispare un slash (como Terra Blade), usar `item.useStyle = ItemUseStyleID.Thrust` para estocada (lanzas cortas), o `item.useStyle = ItemUseStyleID.HoldUp` para items rituales. Para "true melee" (espadas que NO disparan proyectiles y solo dañan en contacto), simplemente se omite `item.shoot` y se confía en `OnHitNPC`.

8.2 Espadas que disparan proyectiles

El patrón más común para espadas mid/late game. En SetDefaults, se configura item.shoot con un ProjectileID o ModContent.ProjectileType<T>(), item.shootSpeed para la velocidad inicial, y opcionalmente item.useAmmo si se quiere que consuma munición (raro en espadas). Para que el proyectil salga en la dirección correcta, tModLoader usa automáticamente la dirección del cursor. Para personalizar el patrón (múltiples proyectiles, spread, etc.), se overridea Shoot():

```
public override bool Shoot(Player player, EntitySource_ItemUse_WithAmmo source,
                           Vector2 position, Vector2 velocity, int type,
                           int damage, float knockback)
{
    int numProjectiles = 3; // Disparar 3 proyectiles en spread
    float rotation = MathHelper.ToRadians(15); // 15 grados de spread total
    for (int i = 0; i < numProjectiles; i++)
    {
        Vector2 perturbedSpeed = velocity.RotatedBy(
            MathHelper.Lerp(-rotation / 2, rotation / 2, i /
(float)(numProjectiles - 1)));
        Projectile.NewProjectile(source, position, perturbedSpeed,
                                type, damage, knockback, player.whoAmI);
    }
    return false; // false = cancelar el spawn del proyectil vanilla
}
```

8.3 Yoyos

Los yoyos son armas melee que se sostienen mientras se mantiene pulsado el botón de uso. La mecánica es: el item tiene item.useStyle = ItemUseStyleID.Shoot y item.channel = true, y dispara un proyectil con Projectile.aiStyle especial. La forma correcta en 1.4.4 es usar YoyoProjectile o heredar de él. Stats relevantes: Projectile.localAI[0] almacena el timer de vida del yoyo, y hay un límite basado en player.GetWeaponKnockback. Para extender el alcance, se usan accesorios como Yoyo Bag.

```
public override void SetDefaults()
{
    Item.damage = 40;
    Item.DamageType = DamageClass.Melee;
    Item.useStyle = ItemUseStyleID.Shoot;
    Item.useAnimation = 25;
    Item.useTime = 25;
    Item.shootSpeed = 16f;
    Item.shoot = ModContent.ProjectileType<MiYoyoProjectile>();
    Item.channel = true; // MANTENER pulsado
    Item.noMelee = true; // El item en sí no hace daño, solo el proyectil
    Item.noUseGraphic = true; // No dibujar el item al usar
    Item.knockBack = 4f;
    Item.rare = ItemRarityID.LightRed;
    Item.value = Item.buyPrice(gold: 5);
}
```

8.4 Flails

Las flails (cadenas con bola) usan `item.useStyle = ItemUseStyleID.Shoot + item.channel = true`, y disparan un proyectil con `Projectile.aiStyle = 15` (flail). El proyectil custom debe heredar de `FlailProjectile` o configurar manualmente los campos relevantes: `Projectile.extraUpdates`, `Projectile.alpha`, y el array `Projectile.ai[]` con estado (lanzado, volviendo, etc.). El `ExampleMod` incluye un ejemplo completo de flail en `ExampleFlail.cs` que es la mejor referencia.

8.5 Boomerangs

Los boomerangs son armas melee que se lanzan y regresan. Patrón: `item.useStyle = ItemUseStyleID.Swing`, `item.shoot = ModContent.ProjectileType<MiBoomerang>()`, `item.useAmmo = AmmoID.None`, `item.autoReuse = false` (usualmente). El proyectil usa `Projectile.aiStyle = 3` (boomerang) que gestiona automáticamente el retorno. Para personalizar el patrón de retorno, se puede overridear `AI()` manteniendo la lógica vanilla con `AIType`. Ejemplo: boomerang que deja un trail de partículas.

9. Creación de Armas Ranged

9.1 Sistema de munición

Las armas ranged usan munición consumible. El sistema se basa en dos campos: `Item.useAmmo` indica qué tipo de munición consume el arma (`AmmoID.Arrow`, `AmmoID.Bullet`, `AmmoID.Dart`, `AmmoID.Rocket`, `AmmoID.Sand`, `AmmoID.Flare`, `AmmoID.Snowball`, `AmmoID.Stynger`), e `Item.shoot` indica qué proyectil se dispara cuando se usa el arma. El comportamiento default es: el arma consume 1 unidad de ammo del inventario del jugador y dispara el proyectil definido en el ammo (no en el arma). Si `Item.shoot` es un proyectil específico (no `ProjectileID.None`), el arma dispara SIEMPRE ese proyectil independientemente del ammo. Si `Item.shoot` es `ProjectileID.WoodenArrowFriendly`, el arma convierte flechas normales en un proyectil custom pero respeta las propiedades del ammo especial (Frost Arrow, Fire Arrow, etc.). Este es el patrón del Daedalus Stormbow y muchos arcos custom.

9.2 Arco básico

```
public override void SetDefaults()
{
    Item.damage = 15;
    Item.DamageType = DamageClass.Ranged;
    Item.width = 14;
    Item.height = 32;
    Item.useTime = 28;
    Item.useAnimation = 28;
    Item.useStyle = ItemUseStyleID.Shoot;
    Item.knockBack = 2f;
    Item.value = Item.buyPrice(silver: 25);
    Item.rare = ItemRarityID.White;
    Item.UseSound = SoundID.Item5; // Sonido de arco
    Item.autoReuse = true;
    Item.useAmmo = AmmoID.Arrow;
    Item.shoot = ProjectileID.WoodenArrowFriendly; // Respeta ammo especial
    Item.shootSpeed = 7f;
    Item.useTurn = false;
    Item.noMelee = true; // El arma no hace daño melee
}
```

9.3 Gun básico

Las guns son casi idénticas a los arcos pero usan AmmoID.Bullet y SoundID.Item11 (sonido de disparo genérico). Ejemplo de pistola básica:

```
public override void SetDefaults()
{
    Item.damage = 18;
    Item.DamageType = DamageClass.Ranged;
    Item.width = 40;
    Item.height = 20;
    Item.useTime = 15;
    Item.useAnimation = 15;
    Item.useStyle = ItemUseStyleID.Shoot;
    Item.knockBack = 3f;
    Item.value = Item.buyPrice(gold: 1);
    Item.rare = ItemRarityID.Blue;
    Item.UseSound = SoundID.Item11;
    Item.autoReuse = true;
    Item.useAmmo = AmmoID.Bullet;
    Item.shoot = ProjectileID.PurificationPowder; // Dispara bullets sin
conversion
    Item.shootSpeed = 10f;
    Item.noMelee = true;
}
```

9.4 Munición custom

Para crear munición custom (flechas, balas, etc.), se crea un ModItem con Item.ammo asignado a un AmmoID y Item.shoot configurado con el proyectil custom. La munición es consumible por defecto (Item.consumable = true, Item.maxStack = 999). Ejemplo de flecha explosiva:

```
public override void SetStaticDefaults()
{
    DisplayName.SetDefault("Flecha Explosiva");
}
```

```

public override void SetDefaults()
{
    Item.damage = 12;
    Item.DamageType = DamageClass.Ranged;
    Item.width = 8;
    Item.height = 8;
    Item.maxStack = 999;
    Item.consumable = true;
    Item.knockBack = 1.5f;
    Item.value = 10;
    Item.rare = ItemRarityID.Green;
    Item.shoot = ModContent.ProjectileType<FlechaExplosivaProjectile>();
    Item.shootSpeed = 3f;
    Item.ammo = AmmoID.Arrow; // Es flecha
}

```

9.5 Patrón Daedalus Stormbow

El Daedalus Stormbow dispara flechas desde el cielo, no desde el jugador. Para replicar este patrón, se overridea Shoot() en el arma y se spawna el proyectil en una posición elevada:

```

public override bool Shoot(Player player, EntitySource_ItemUse_WithAmmo source,
                           Vector2 position, Vector2 velocity, int type,
                           int damage, float knockback)
{
    Vector2 target = Main.MouseWorld;
    for (int i = 0; i < 3; i++)
    {
        Vector2 spawnPos = new Vector2(
            target.X + Main.rand.Next(-200, 200),
            target.Y - 600 - Main.rand.Next(0, 200)
        );
        Vector2 vel = (target - spawnPos) * 0.02f;
        Projectile.NewProjectile(source, spawnPos, vel, type, damage, knockback,
            player.whoAmI);
    }
    return false;
}

```

10. Creación de Armas Magic

10.1 Configuración básica

Las armas mágicas comparten el patrón ranged (useStyle Shoot, noMelee true) pero añaden Item.mana para el coste de maná. La reducción de maná por accesorios se aplica automáticamente. El daño se escala con player.GetDamage<MagicDamageClass>() (o DamageClass.Magic en la API legacy). Ejemplo:

```

public override void SetDefaults()
{
    Item.damage = 22;
    Item.DamageType = DamageClass.Magic;
    Item.mana = 6;
    Item.width = 28;
    Item.height = 30;
    Item.useTime = 18;
}

```

```

Item.useAnimation = 18;
Item.useStyle = ItemUseStyleID.Shoot;
Item.knockBack = 4f;
Item.value = Item.buyPrice(gold: 2);
Item.rare = ItemRarityID.Green;
Item.UseSound = SoundID.Item8;
Item.autoReuse = true;
Item.shoot = ModContent.ProjectileType<MiMagicProjectile>();
Item.shootSpeed = 12f;
Item.noMelee = true;
}

```

10.2 Patrones de disparo

Magic weapons suelen tener patrones más elaborados que ranged. Algunos patrones comunes implementados overrideando Shoot(): spread de N proyectiles en abanico (ver §8.2), proyectiles que se dividen al impactar, proyectiles sinusoidales (movimiento ondulado), proyectiles que orbitan al jugador antes de salir, y proyectiles con carga (mantener pulsado para aumentar daño). Ejemplo de proyectil sinusoidal:

```

// En ModProjectile.AI()
public override void AI()
{
    Projectile.ai[0]++;
    // Movimiento sinusoidal perpendicular a la velocidad inicial
    Vector2 perpendicular = new Vector2(-Projectile.velocity.Y,
    Projectile.velocity.X);
    perpendicular.Normalize();
    float offset = (float)Math.Sin(Projectile.ai[0] * 0.2f) * 5f;
    Projectile.position += perpendicular * offset * 0.1f;

    // Spawn de partículas
    if (Main.rand.NextBool(3))
    {
        Dust.NewDust(Projectile.position, Projectile.width, Projectile.height,
            DustID.MagicMirror, 0, 0, 150, default, 1f);
    }
}

```

10.3 Sistema de carga (charge-up)

Para un arma que se carga manteniendo pulsado el botón: usar Item.channel = true para que el ítem no se recargue hasta soltar, overridear HoldItem() para llevar un contador (almacenado en player.GetModPlayer), y overridear Shoot() o UseItem() para ejecutar el disparo final. La lógica del contador debe vivir en un ModPlayer para no perderse entre frames.

11. Creación de Armas Summon

11.1 Tipos de armas summon

Hay tres subtipos de armas summon: minions (criaturas que siguen al jugador y atacan), sentries (torres estacionarias), y whips (armas cuerpo a cuerpo que aplican "summon tag damage" para que los minions hagan más daño al objetivo golpeado). Cada subtipo tiene su propio patrón de implementación.

11.2 Minions

Un minion es un Projectile con `Projectile.minion = true` y `Projectile.minionSlots = X` (X = cuántos slots consume del jugador). El jugador tiene `player.maxMinions` slots (1 base + bonus por accesorios/armadura). Al usar un item summon, se invoca el minion y se aplica un buff que lo mantiene vivo mientras el jugador tenga slots disponibles. Implementación típica:

```
// Item summon
public override void SetDefaults()
{
    Item.damage = 20;
    Item.DamageType = DamageClass.Summon;
    Item.mana = 10;
    Item.width = 26;
    Item.height = 28;
    Item.useTime = 36;
    Item.useAnimation = 36;
    Item.useStyle = ItemUseStyleID.Swing;
    Item.noMelee = true;
    Item.knockBack = 2f;
    Item.value = Item.buyPrice(gold: 5);
    Item.rare = ItemRarityID.Pink;
    Item.UseSound = SoundID.Item44;
    Item.shoot = ModContent.ProjectileType<MiMinion>();
    Item.buffType = ModContent.BuffType<MiMinionBuff>(); // Asignado
    automáticamente
}

public override bool Shoot(Player player, EntitySource_ItemUse_WithAmmo source,
    Vector2 position, Vector2 velocity, int type,
    int damage, float knockback)
{
    player.AddBuff(Item.buffType, 2); // Buff infinito (se refresca)
    Projectile.NewProjectile(source, position, velocity, type, damage,
        knockback, player.whoAmI);
    return false;
}

// Projectile minion
public override void SetStaticDefaults()
{
    Main.projPet[Type] = true; // Es un minion
    ProjectileID.Sets.MinionTargettingFeature[Type] = true;
}

public override void SetDefaults()
{
    Projectile.minion = true;
    Projectile.minionSlots = 1f; // Consume 1 slot
    Projectile.friendly = true;
```

```

Projectile.penetrate = -1; // Infinito
Projectile.width = 20;
Projectile.height = 20;
Projectile.tileCollide = false; // Atraviesa paredes
Main.projFrames[Type] = 4; // 4 frames de animación
}

public override void AI()
{
    Player player = Main.player[Projectile.owner];
    // Verificar si el buff está activo (si no, desaparecer)
    if (!player.HasBuff(ModContent.BuffType<MiMinionBuff>()))
    {
        Projectile.Kill();
        return;
    }

    // Lógica: seguir al jugador, buscar target, atacar
    Vector2 idlePos = player.Center + new Vector2(0, -60);
    float maxDist = 800f;

    // Buscar target
    NPC target = null;
    foreach (NPC npc in Main.npc)
    {
        if (!npc.active || !npc.CanBeChasedBy(Projectile)) continue;
        float dist = Vector2.Distance(npc.Center, player.Center);
        if (dist < maxDist)
        {
            maxDist = dist;
            target = npc;
        }
    }

    if (target != null)
    {
        Vector2 toTarget = target.Center - Projectile.Center;
        toTarget.Normalize();
        Projectile.velocity = Vector2.Lerp(
            Projectile.velocity, toTarget * 8f, 0.1f);
    }
    else
    {
        // Volver al jugador
        Vector2 toPlayer = idlePos - Projectile.Center;
        if (toPlayer.Length() > 100f)
        {
            toPlayer.Normalize();
            Projectile.velocity = Vector2.Lerp(
                Projectile.velocity, toPlayer * 6f, 0.05f);
        }
    }
}

```

11.3 Sentries

Las sentries son similares a los minions pero estacionarias: `Projectile.sentry = true` en `SetDefaults`, y en `AI()` el projectile no se mueve, solo rota/dispara. El item summon de sentry usa `Item.UseStyle = ItemUseStyleID.HoldUp`. Las sentries consumen `player.maxTurrets` slots en lugar de `maxMinions`.

11.4 Whips

Los whips usan `DamageClass.SummonMeleeSpeed`, que escala con `melee speed` (por lo que accesorios como `Feral Claws` aumentan la velocidad de uso del whip). Su mecánica única es aplicar `SummonTagDamage` al NPC golpeado, lo que hace que los minions del jugador hagan más daño a ese NPC específico durante unos segundos. Implementación simplificada:

```
public override void SetDefaults()
{
    Item.damage = 18;
    Item.DamageType = DamageClass.SummonMeleeSpeed;
    Item.width = 38;
    Item.height = 36;
    Item.useTime = 30;
    Item.useAnimation = 30;
    Item.useStyle = ItemUseStyleID.Swing;
    Item.knockBack = 4f;
    Item.value = Item.buyPrice(gold: 3);
    Item.rare = ItemRarityID.Green;
    Item.UseSound = SoundID.Item152;
    Item.autoReuse = true;
    Item.shoot = ModContent.ProjectileType<MiWhipProjectile>();
    Item.shootSpeed = 4f;
    Item.noMelee = true;
    Item.noUseGraphic = true;
}

public override void OnHitNPC(Player player, NPC target,
                              int damage, float knockBack, bool crit)
{
    // Aplicar summon tag damage
    var tagPlayer = player.GetModPlayer<WhipTagPlayer>();
    tagPlayer.ApplyTag(target, 4); // +4 daño extra de minions por 4 segundos
}
```

12. Creación de Armas Throwing/Rogue

12.1 Estado de la clase Throwing

En Terraria 1.3, "throwing" era una clase de daño completa (shurikens, throwing knives, etc.). En 1.4 vanilla se eliminó como clase propia y los items throwing pasaron a ser ranged. Sin embargo, tModLoader mantiene `DamageClass.Throwing` en la API para que los modders puedan seguir usándolo. El mod "Return of the Thrower" (Steam Workshop, ID 2585923219) re-implementa la clase thrower convirtiendo items vanilla a throwing damage. La clase Rogue de Calamity es otra implementación custom que añade mecánicas de stealth.

12.2 Item throwing básico

```
public override void SetDefaults()
{
    Item.damage = 15;
    Item.DamageType = DamageClass.Throwing;
    Item.width = 14;
    Item.height = 14;
    Item.maxStack = 999;
    Item.consumable = true; // Se consume al usar
    Item.useTime = 15;
    Item.useAnimation = 15;
    Item.useStyle = ItemUseStyleID.Swing;
    Item.knockBack = 1f;
    Item.value = 5;
    Item.rare = ItemRarityID.Blue;
    Item.UseSound = SoundID.Item1;
    Item.autoReuse = true;
    Item.shoot = ModContent.ProjectileType<ShurikenExplosivoProjectile>();
    Item.shootSpeed = 10f;
    Item.noUseGraphic = true; // No dibujar el item al lanzar
}
```

12.3 Clase Rogue de Calamity

La Rogue class de Calamity es el ejemplo canónico de DamageClass custom. Añade la mecánica de "stealth": al no atacar durante un tiempo, el jugador acumula stealth que se consume al lanzar un arma rogue, amplificando el daño y/o el tamaño del proyectil. La implementación usa un ModPlayer que trackea el stealth meter, un ModProjectile que en AI() verifica la stealth del jugador para aplicar bonificadores, y una DamageClass custom RogueDamageClass. Para estudiar el código completo, ver el archivo CalamityMod/Classes/RogueDamageClass.cs en el [mirror público](#).

13. Sistema de Accesorios

13.1 ModItem como accesorio

Cualquier ModItem puede ser accesorio estableciendo Item.accessory = true en SetDefaults. El hook clave es UpdateAccessory(Player player, bool hideVisual), que se ejecuta cada frame mientras el accesorio está equipado. Aquí se modifican las stats del jugador: player.statDefense, player.moveSpeed, player.magicDamage, player.meleeDamage, player.rangedDamage, player.minionDamage, player.manaCost, player.maxMinions, player.lifeRegen, player.endurance, etc. El parámetro hideVisual indica si el jugador activó "hide visual" en su slot de accesorio (no afecta a la lógica, solo al rendering). Ejemplo de accesorio de melee:

```
public override void SetStaticDefaults()
{
    DisplayName.SetDefault("Emblema del Berserker");
    Tooltip.SetDefault("+10% melee damage\n+5% melee speed");
}

public override void SetDefaults()
{
    Item.width = 28;
```

```

        Item.height = 28;
        Item.value = Item.buyPrice(gold: 2);
        Item.rare = ItemRarityID.LightRed;
        Item.accessory = true;
    }

    public override void UpdateAccessory(Player player, bool hideVisual)
    {
        player.GetDamage(DamageClass.Melee) += 0.10f;
        player.GetAttackSpeed(DamageClass.Melee) += 0.05f;
    }

```

13.2 ModPlayer para efectos custom

Para efectos que no caben en UpdateAccessory (cooldowns, stacks, estado complejo), se usa ModPlayer. Esta clase se ejecuta para cada jugador y expone hooks como PreUpdate(), PostUpdate(), OnHitAnything(), OnHitNPCWithItem(), OnHitNPCWithProj(). El accesorio activa un flag en ModPlayer, y ModPlayer aplica el efecto en el hook apropiado. Para invocaciones múltiples de accesorios que stackean, usar un contador en ModPlayer que se resetea cada frame en PreUpdate y se incrementa desde cada UpdateAccessory.

14. Sistema de Recetas

14.1 Recipe.Create API

En tModLoader 1.4, las recetas se crean con Recipe.Create() en AddRecipes(). El patrón es:

```

public override void AddRecipes()
{
    Recipe.Create(ModContent.ItemType<MiEspada>())
        .AddIngredient(ItemID.WoodenSword, 1)
        .AddIngredient(ItemID.GoldBar, 5)
        .AddIngredient(ModContent.ItemType<MiMaterial>(), 3)
        .AddTile(TileID.Anvils)
        .Register();

    // Segunda receta con ingredientes diferentes
    Recipe.Create(ModContent.ItemType<MiEspada>())
        .AddIngredient(ItemID.WoodenSword, 1)
        .AddIngredient(ItemID.PlatinumBar, 5)
        .AddTile(TileID.Anvils)
        .Register();
}

```

14.2 Recipe Groups

Los Recipe Groups permiten agrupar varios items equivalentes (ej: cualquier lingote de tier 1 = cobre o estaño). Se registran en ModSystem.OnModLoad() con RecipeGroup.RegisterGroup():

```

// En ModSystem.OnModLoad()
RecipeGroup.RegisterGroup("MiMod:LingotesTier1", new RecipeGroup(() =>
    "Cualquier lingote de tier 1",
    ItemID.CopperBar, ItemID.TinBar, ItemID.IronBar, ItemID.LeadBar

```

```
));

// En AddRecipes()
Recipe.Create(ModContent.ItemType<MiEspada>())
    .AddRecipeGroup("MiMod:LingotesTier1", 5)
    .AddTile(TileID.Anvils)
    .Register();
```

14.3 Cross-mod en recetas

Para añadir ingredientes de otro mod condicionalmente (cross-mod), se usa `ModContent.TryFind` o `ModLoader.TryGetMod`. Esto es seguro: si el mod referenciado no está, la receta simplemente no se registra.

```
public override void AddRecipes()
{
    if (ModLoader.TryGetMod("CalamityMod", out Mod calamity))
    {
        if (calamity.TryFind("Bloodstone", out ModItem bloodstone))
        {
            Recipe.Create(ModContent.ItemType<MiArmaAvanzada>())
                .AddIngredient(bloodstone, 10)
                .AddTile(TileID.LunarCraftingStation)
                .Register();
        }
    }
}
```

15. Catálogo de Repositorios de GitHub

A continuación se detalla cada repositorio relevante encontrado durante la investigación, ordenado por relevancia para un modder de armas. Para cada repo se incluye URL, autor, descripción, características destacadas y por qué vale la pena estudiarlo.

15.1 tModLoader (framework base)

Campo	Valor
URL	https://github.com/tModLoader/tModLoader
Autor	Organización tModLoader
Licencia	MIT
Lenguaje	C# / .NET 6+
Versión	1.4.4 stable
Stars	~4.5k+ (crecimiento activo)
¿Incluye ExampleMod?	Sí, en rama 1.4.4

El repositorio principal de tModLoader. Imprescindible: contiene el source completo del modding API, el ExampleMod (en ExampleMod/ folder de la rama 1.4.4) que es la mejor referencia para aprender patrones, y el wiki con tutoriales oficiales. Los Issues del repo son una mina de oro para entender edge cases (#1173 DamageClass overhaul, #3376 arkhalis-style weapons, #3439 spinner weapons, #1223 projectile control from items, #3369 boomerang errors). Wiki: <https://github.com/tModLoader/tModLoader/wiki>

15.2 CalamityModPublic (mirror del mod más grande)

Campo	Valor
URL	https://github.com/CalamityTeam/CalamityModPublic
Autor	CalamityTeam (Azafure LLC)
Licencia	Código público, assets bajo copyright
Lenguaje	C# / .NET 6+
Versión	2.x (Calamity 1.4.4)
Tamaño	Miles de armas, jefes, biomas, mecánicas
¿Actualizado?	Sí, sincronizado con Steam Workshop

Mirror público del mod más famoso de Terraria. Es el mejor recurso para aprender patrones avanzados: armas con comportamientos complejos (espadas con 5 fases, guns con charge-up, magic weapons con proyectiles que se dividen), implementación completa de la clase Rogue custom, sistemas de configuración, bosses con fases, y compatibilidad cross-mod. Organización GitHub: github.com/CalamityTeam. Wiki oficial: calamitymod.wiki.gg. La comunidad en Reddit lo considera "el mejor recurso de aprendizaje para modding avanzado" (ver hilo [r/CalamityMod](https://www.reddit.com/r/CalamityMod) sobre el mirror público).

15.3 ThoriumMod

Campo	Valor
URL	https://github.com/SamsonAllen13/ThoriumMod
Autor	Samson Allen / DivermanSam
Licencia	Código cerrado, repo solo para releases
Versión	1.4.4 (port en progreso)
Tamaño	2.600+ items, 11 clases, 3 clases nuevas

Uno de los mods "Vanilla+" más grandes. A diferencia de Calamity, el código fuente NO es público: el repo GitHub existe principalmente para distribuir releases y aliviar el ancho de banda del Mod Browser. Sin embargo, los releases incluyen el .tmod que puede ser descompilado con herramientas como tModLoader decompiler para inspección educativa (sujeto a las restricciones de licencia). Steam Workshop ID: 2909886416.

15.4 WeaponEnchantments

Campo	Valor
URL	https://github.com/andro951/WeaponEnchantments
Autor	andro951
Enfoque	Sistema de encantamientos de armas/armor/accessories
Mecánica única	Infundir armas para transferir progresión

Implementa un sistema de XP y encantamientos que se aplica a TODAS las armas, armor y accesorios (incluidos los de otros mods). Es un ejemplo excelente de cómo usar GlobalItem (en lugar de ModItem) para modificar comportamiento de items que no son tuyos. La mecánica de "infundir" permite transferir el XP de un arma obsoleta a una nueva, manteniendo progresión. Útil para estudiar: GlobalItem hooks, sistemas de configuración, UI custom en el inventario.

15.5 AmmoboxPlus

Campo	Valor
URL	https://github.com/Muzuwi/AmmoboxPlus
Autor	Muzuwi
Enfoque	Munición ranged (43 tipos nuevos)
Clase	Ranger (ranged especializado)

Mod enfocado exclusivamente en añadir munición ranged. Cada tipo de ammo tiene efectos únicos: split en múltiples proyectiles, homing, explosión al impacto, debuffs, etc. Es la mejor referencia para aprender patrones de ammo custom y cómo los proyectiles pueden comportarse diferente según el ammo que los generó.

15.6 TerraScape

Campo	Valor
URL	https://github.com/Cyxio/TerraScape
Autor	Cyxio
Tamaño	394 items, 70 armas (22 melee, 10 ranged, 37 magic, 1 throwing)
Armor sets	14 sets + 95 vanity items

Mod equilibrado con distribución de armas por clase. Útil para estudiar cómo organizar un mod mediano con variedad balanceada entre melee/ranged/magic. Los 14 sets de armadura están bien documentados y sirven como referencia para crear sets custom con set bonuses.

15.7 ArknightsMod

Campo	Valor
URL	https://github.com/cueVs/ArknightsMod
Autor	cueVs
Tamaño	500+ armas (magic, ranged, melee, summon)
Tema	Crossover con Arknights

Mod temático de Arknights con 500+ armas distribuidas entre las 4 clases principales. Útil para ver cómo estructurar un mod grande con identidad temática fuerte (cada arma está basada en un operador de Arknights) y cómo mantener coherencia visual/mechanic en un proyecto extenso.

15.8 Persona5Cosplay

Campo	Valor
URL	https://github.com/Huckletoon/Persona5Cosplay
Autor	Huckletoon
Tema	Persona 5 / SMT crossover
Enfoque	Armas y armaduras temáticas por personaje

Implementa armas temáticas del universo Persona/SMT: magic spells estilo Agi/Bufula/Megido, melee por party member, ranged con guns temáticas. Útil para estudiar cómo organizar un mod pequeño-mediano alrededor de una temática narrativa y cómo estructurar sets de armadura por personaje.

15.9 CalamityEntropy (addon de Calamity)

Campo	Valor
URL	https://github.com/WarmurCle/CalamityEntropy
Autor	hochal13
Tipo	Addon de Calamity
Añade	Armas, armors, accessories, bosses, música

Ejemplo canónico de cómo escribir un addon para Calamity. El código referencia implementaciones de Calamity y añade contenido que depende de él. Útil para aprender: patrones de cross-mod con dependencia fuerte (modReferences en build.txt), cómo extender la clase Rogue de Calamity, cómo añadir recetas que usan materiales de Calamity.

15.10 AlchemistNPC

Campo	Valor
URL	https://github.com/VVV101/AlchemistNPC
Autor	VVV101
Tipo	NPC vendor + thrown weapons
Característica	Stormbreaker como thrown weapon

Mod centrado en NPCs vendedores (Alchemist, Brewer, Jeweler, Operator, etc.) que venden items útiles incluyendo armas. Es un buen ejemplo de cómo implementar ModNPC con Town NPC behaviour, shop UI, y cómo aplicar modifiers fijos (Legendary para melee, Unreal para ranged) a items vendidos. También incluye ejemplos de thrown weapons (Stormbreaker).

15.11 ie53-Terraria-All-Items-Mod

Campo	Valor
URL	https://github.com/ie53-Terraria-All-Items-Mod
Autor	ie53
Tipo	Utility mod (chest con todos los items)
Filtro	Por categoría (melee, ranged, magic, etc.)

Mod utilidad para testing: añade un chest/NPC que da acceso a todos los items del juego y de mods cargados. Útil para desarrolladores que quieren testear sus armas sin grind. También sirve como ejemplo de cómo enumerar items de otros mods programáticamente (`ModContent.GetContent<ModItem>()`).

15.12 tModLoader ExampleMod

Campo	Valor
URL	https://github.com/tModLoader/tModLoader/tree/1.4.4/ExampleMod
Autor	Equipo tModLoader (oficial)
Versión	Sincronizada con tModLoader 1.4.4
Steam Workshop	ID 2615761271

El mod de referencia oficial. Incluye ejemplos comentados de: espadas (ExampleSword), arcos (ExampleBow), guns (ExampleGun), magic weapons (ExampleMagicWeapon), minions (ExampleMinion), whips (ExampleWhip), flails (ExampleFlail), yoyos (ExampleYoyo), accesorios (ExampleAccessory), NPCs (ExampleNPC), tiles (ExampleTile), y systems (ExampleModSystem). Cada archivo está densamente comentado explicando cada línea. Es el primer lugar donde un desarrollador nuevo debería mirar.

15.13 Otros repos menores y recursos

Adicionalmente, existen otros recursos relevantes: github.com/topics/tmodloader-mod (listado completo de todos los mods con código abierto en GitHub), github.com/topics/calamitymod (addons de Calamity), github.com/MountainDrew8/CalamityMod (legacy releases pre-mirror público), github.com/SpaghettiLord1010/TModMaker (herramienta para generar mods visualmente), github.com/SaerusTierialis/tModLoader_ExperienceAndClasses (sistema RPG de clases y leveling). Para mods adicionales, el Mod Browser in-game (mirror legacy en mirror.sgkoi.dev) lista cientos de mods con descripción y categoría.

16. Análisis Comparativo de Arquitecturas

16.1 Organización por clase de daño (Calamity)

Calamity organiza su carpeta `Items/` con subcarpetas por clase de daño: `Items/Melee/`, `Items/Ranged/`, `Items/Magic/`, `Items/Summon/`, `Items/Rogue/`. Dentro de cada subcarpeta, los items se agrupan por subtipo (`Swords/`, `Yoyos/`, `Spears/` dentro de `Melee/`). Esta organización facilita encontrar items por clase pero puede llevar a carpetas muy pobladas (Calamity tiene cientos de archivos por subcarpeta). Para mods grandes con >100 armas, es el patrón recomendado.

16.2 Organización por tipo de contenido (ExampleMod)

ExampleMod usa organización por tipo: `Content/Items/Weapons/`, `Content/Items/Accessories/`, `Content/Items/Materials/`, `Content/Projectiles/`, `Content/NPCs/`, `Content/Tiles/`. Este patrón es más natural para mods pequeños-medianos (<100 items) y es el que usa la mayoría de mods comunitarios. La ventaja es que encontrar "todas las armas" es trivial; la desventaja es que no separa por clase, lo que en mods grandes se vuelve confuso.

16.3 Organización por tier de progresión (Thorium)

Thorium usa una organización híbrida donde los items se agrupan por tier de progresión dentro de cada clase: `PreHM/Tier1/`, `PreHM/Tier2/`, `Hardmode/Tier1/`, etc. Esto facilita el balance (todos los items del mismo tier están juntos) pero dificulta encontrar items específicos. Es un patrón avanzado que solo recomendamos para mods muy grandes con balance cuidadoso por tier.

16.4 Buenas prácticas transversales

- Usar `[ContentSortOrder]` attribute para controlar el orden de carga de `ModSystems`
- Usar `[Autoload]` y `[AutoloadHead/Etc]` en lugar de `Mod.AddContent()` manual
- `ModContent.GetInstance<T>()` para singletons (configuraciones, datos globales)
- Nombrar archivos `.cs` exactamente igual que la clase pública que contienen

- Mantener namespaces consistentes con la estructura de carpetas
- Una clase por archivo (no meter 5 ModItems en un solo .cs)
- Usar ModSystem para lógica compartida entre múltiples items/projectiles
- Comentar código denso (especialmente AI() de proyectiles complejos)

16.5 Anti-patrones comunes

- Clases gigantes con miles de líneas (split en métodos privados o clases helper)
- Repetir código en lugar de usar herencia (clase base abstracta con lógica común)
- Lógica compartida en ModItem/ModProjectile en lugar de ModSystem o GlobalItem
- Variables estáticas para estado de instancia (causa desync multiplayer)
- Sprites PNG >4096px (causa OutOfMemoryException en carga)
- Magic numbers en AI() en lugar de constantes con nombre

17. Distribución y Publicación

17.1 Pipeline de build y test

El flujo estándar para preparar un mod para publicación es: 1) Editar código C# en Visual Studio, 2) En tModLoader in-game ir a Workshop -> Develop Mods, seleccionar el mod, pulsar "Build + Reload", 3) tModLoader recompila el .csproj y carga el mod en caliente, 4) Test in-game (idealmente en multiplayer también), 5) Si hay errores de compilación, leer la consola in-game y el log en Documents/My Games/Terraria/tModLoader/Logs/, 6) Iterar hasta que no haya errores, 7) Cuando el mod está estable, bump de versión en build.txt y publicar.

17.2 Publicar en Steam Workshop

Para publicar en Steam Workshop: 1) En tModLoader ir a Workshop -> Publish Mods, 2) Seleccionar tu mod de la lista, 3) Configurar metadatos: nombre público, descripción (Markdown soportado), tags (Weapons, Content, QoL, etc.), 4) Subir icon.png (256x256 px), 5) Subir screenshots (recomendado al menos 3), 6) Configurar visibilidad (Public, Friends Only, Private), 7) Click "Publish". El mod recibirá un Workshop ID único que se usa para suscribirse. Las actualizaciones se hacen desde el mismo menú: bump de versión en build.txt, rebuild, click "Update" en el menú de Workshop.

17.3 Mod References en build.txt

Si tu mod depende de otro mod (ej: un addon de Calamity), se declaran en build.txt con modReferences = CalamityMod. tModLoader no cargará tu mod si CalamityMod no está presente. Para dependencias opcionales (cross-mod content condicional), se usa weakReferences = CalamityMod, que permite al mod cargar pero sin el contenido cross-mod si Calamity no está. La versión del mod referenciado se puede especificar con @ (ej: modReferences = CalamityMod @ 2.0.4.5).

17.4 GitHub Releases como mirror

Aunque Steam Workshop es el canal principal, muchos modders mantienen un GitHub repo con releases binarias del .tmod file. Esto sirve como: backup, canal para versiones beta/preview, forma de distribuir a usuarios de GOG (que no tienen Steam Workshop), y para citación académica o referencia. El patrón típico es taggear cada release con la versión (v1.0.0, v1.0.1) y adjuntar el .tmod como asset. GitHub Actions puede automatizar este proceso on tag push.

17.5 Versionado semántico

Se recomienda seguir Semantic Versioning (semver.org): MAJOR.MINOR.PATCH. MAJOR para cambios incompatibles (rompen saves o compatibilidad con otros mods), MINOR para nuevo contenido compatible, PATCH para bug fixes. El campo version en build.txt se actualiza en cada release. Cuando se actualiza un mod ya publicado, el bump de versión es OBLIGATORIO: Steam Workshop rechaza updates sin bump de versión. La convención entre modders es bump de PATCH para fixes rápidos, MINOR para añadir contenido, MAJOR para grandes reworks.

18. Pitfalls Comunes y Debugging

18.1 NullReferenceException en SetDefaults

Causa típica: intentar acceder a `Main.player[Projectile.owner]` u otros arrays antes de que estén inicializados. Solución: en `SetDefaults`, no acceder a estado del mundo/jugador; solo inicializar campos del propio item/projectile. Si necesitas lógica que depende del jugador, muévela a `OnSpawn` o `AI()`.

18.2 Projectile no aparece al disparar

Causas comunes: (a) `Item.shoot` no configurado o `= ProjectileID.None`, (b) `Projectile.aiStyle` incorrecto para el `ModProjectile`, (c) `projectile.friendly = false` (los proyectiles no friendly no dañan NPCs), (d) `projectile.hostile = true` accidental, (e) en multiplayer, `Projectile.NewProjectile` sin owner correcto. Solución: revisar `SetDefaults` del `ModProjectile` y verificar que `friendly=true`, `hostile=false`, y que el type del `item.shoot` es correcto.

18.3 Sprite no carga / aparece como cuadrado rojo

Causa típica: el path del PNG no coincide con namespace+classname. Si tu clase es `MiMod.Content.Items.Weapons.EspadaX`, `tModLoader` busca `MiMod/Content/Items/Weapons/EspadaX.png`. Si el archivo está en otra ubicación, overridear la propiedad `Texture` en la clase. Otras causas: PNG >4096px en cualquiera dimensión, PNG en formato incorrecto (debe ser RGBA 32-bit), archivo .png en minúscula cuando la clase está en mayúscula (en Linux/macOS esto importa porque el filesystem es case-sensitive).

18.4 Daño no aplica correctamente

Causa más común: `Item.DamageType` no asignado en `SetDefaults`. Sin `DamageType`, el item cae en `DamageClass.Default` y no recibe bonificadores de daño del jugador (un accesorio +10% melee no afectará al item). Otra causa: `Projectile.friendly = false` (los proyectiles no friendly no dañan NPCs). También: `Projectile.DamageType` no coincide con `Item.DamageType` (los proyectiles heredan por defecto pero se puede overridear).

18.5 Multiplayer desync

El problema más difícil de debuggear. Síntomas: el mod funciona en singleplayer pero en multiplayer ciertas mecánicas no se sincronizan, o los clientes ven comportamientos diferentes al host. Causa raíz: usar variables estáticas o campos de instancia para estado que debería estar en `Projectile.ai[]` o en un `ModPlayer`. Solo `Projectile.ai[0]` y `Projectile.ai[1]` se sincronizan automáticamente. Para estado custom del jugador, usar `ModPlayer` con fields públicos (tModLoader los sincroniza si la clase tiene el atributo correcto). Para estado custom de proyectiles que no cabe en `ai[]`, usar `ModNet.Send/ReceivePacket`.

18.6 OutOfMemoryException al cargar

Causa: texturas PNG excesivamente grandes (>4096px en cualquier dimensión), o demasiados items precargados. Solución: mantener sprites ≤1024px, usar `Main.QueueMainThreadAction` para cargar assets bajo demanda, y evitar `Main.assetRequestStack` creciente. El log mostrará el archivo problemático. Si el mod tiene miles de items con sprites grandes, considerar comprimir o usar atlas.

18.7 Build errors comunes

Error	Causa	Solución
CS0246: type or namespace not found	Falta using o dllReference	Añadir using <code>Terraria.ModLoader</code> ; o <code>dllReferences</code> en <code>build.txt</code>
CS0117: type does not contain definition	API cambió entre versiones	Verificar que usas la API de 1.4.4, no 1.3
CS0103: name does not exist in current context	Falta using o typo	Verificar namespaces y nombres exactos
Autoload failure	Missing [Texture] o path incorrecto	Overridear propiedad <code>Texture</code> o renombrar archivo
ReflectionTypeLoadException	dllReferences roto	Verificar que la DLL existe y es compatible .NET 6

18.8 Cómo leer los logs

Los logs de tModLoader están en Documents/My Games/Terraria/tModLoader/Logs/ (Windows). Archivos principales: client.log (cliente), server.log (servidor dedicado), launch.log (startup). Cada sesión crea un archivo nuevo con timestamp. Para debuggear un crash, abrir el log más reciente y buscar líneas con "ERROR" o "FATAL". Las excepciones muestran stack trace completo con archivo y línea. Para logs propios, usar `Logging.PublicInfo.WriteLine("mensaje")` (que escribe al log) o `Mod.Logger.Info("mensaje")`.

19. Patrones de Cross-Mod Compatibility

19.1 Mod Calls (sistema de mensajería)

El sistema de Mod Calls permite invocar métodos de otros mods sin tener referencia directa a su código. El mod destino define un método `Call(object[])` que recibe argumentos y devuelve un `object`. El mod origen usa `ModLoader.TryGetMod("OtroMod", out Mod otro)` y luego `otro.Call("mensaje", args)`. Este patrón es seguro: si el mod destino no está, `TryGetMod` devuelve false y se puede omitir la llamada. Es la forma recomendada de cross-mod cuando no se quiere añadir `modReferences`.

19.2 TryFind para items y projectiles

Para acceder a items de otro mod sin dependencia fuerte, se usa `ModLoader.TryGetMod + mod.TryFind`. Ejemplo: si quieres crear una receta que use "Bloodstone" de Calamity (si está instalado), pero tu mod también funciona sin Calamity, el patrón es:

```
public override void AddRecipes()
{
    var recipe = Recipe.Create(ModContent.ItemType<MiArma>())
        .AddIngredient(ItemID.Wood, 10);

    if (ModLoader.TryGetMod("CalamityMod", out Mod calamity) &&
        calamity.TryFind("Bloodstone", out ModItem bloodstone))
    {
        recipe.AddIngredient(bloodstone, 5);
    }

    recipe.AddTile(TileID.WorkBenches)
        .Register();
}
```

19.3 Weak references en build.txt

Si tu mod tiene contenido significativo que depende de otro mod (pero quieres que el mod funcione sin él), usa `weakReferences` en `build.txt`. Esto permite que el mod cargue sin la dependencia pero activa el contenido cross-mod cuando la dependencia está presente. Patrón:

```
# build.txt
weakReferences = CalamityMod @ 2.0.4.5
modReferences =
```

En el código, usar `#if` preprocessor directives para compilar condicionalmente el código cross-mod, o usar `reflection` para invocar tipos del mod referenciado sin `import` directo. Para detalles avanzados, ver el source de Calamity Community Remix que tiene patrones extensivos de weak references.

19.4 Compat con mods utilitarios

Algunos mods utilitarios requieren integración específica para que funcionen bien con tu mod:

- Recipe Browser: si tus recetas usan Recipe Groups custom, registrarlos temprano en `OnModLoad`
- Magic Storage: usar `ModContent.TryFind` para acceder a `StorageHeart` si quieres integrar
- Census Town NPC Checklist: registrar tus Town NPCs via `Mod.Call` con `"AddTownNPC"`
- Helpful NPCs: usar `Call("AddShopItem", item, mod, conditions)` para añadir items a shops
- HEROs Mod: implementar `HEROsModModIntegration` si quieres UI de spawn de items

19.5 Patrón de Mod addon

Un "addon mod" es un mod que existe exclusivamente para extender otro mod (ej: Calamity Entropy para Calamity). El addon usa `modReferences = CalamityMod` en `build.txt` (dependencia fuerte). En el código, puede importar directamente los namespaces de Calamity (using `CalamityMod.Items.Weapons.Melee;`) y heredar de clases de Calamity (ej: `public class MiEspada : CalamityMod.Items.Weapons.CalamityModItem`). El addon debe bump de versión cuando Calamity publica un update breaking, porque los cambios de API pueden romper la compilación.

20. Snippets de Código Reales

Esta sección compila snippets completos y funcionales para los patrones más comunes. Cada snippet es autocontenido y puede copiarse a un archivo `.cs` con los usings apropiados.

20.1 Espada básica con tooltip y receta

```
using Terraria;
using Terraria.ID;
using Terraria.ModLoader;

namespace MiMod.Content.Items.Weapons
{
    public class EspadaHierroMejorada : ModItem
    {
        public override void SetStaticDefaults()
        {
            DisplayName.SetDefault("Espada de Hierro Mejorada");
            Tooltip.SetDefault("Una espada básica con un toque extra de daño");
        }

        public override void SetDefaults()
        {
            Item.damage = 15;
            Item.DamageType = DamageClass.Melee;
            Item.width = 32;
            Item.height = 32;
            Item.useTime = 20;
            Item.useAnimation = 20;
            Item.useStyle = ItemUseStyleID.Swing;
            Item.knockBack = 5f;
            Item.value = Item.buyPrice(silver: 30);
            Item.rare = ItemRarityID.White;
            Item.UseSound = SoundID.Item1;
            Item.autoReuse = true;
        }

        public override void AddRecipes()
        {
            Recipe.Create(ModContent.ItemType<EspadaHierroMejorada>())
                .AddIngredient(ItemID.IronShortsword, 1)
                .AddIngredient(ItemID.IronBar, 5)
                .AddTile(TileID.Anvils)
                .Register();
        }
    }
}
```

20.2 Magic weapon con spread de 5 proyectiles

```
public override bool Shoot(Player player, EntitySource_ItemUse_WithAmmo source,
    Vector2 position, Vector2 velocity, int type,
    int damage, float knockback)
{
    int numProjectiles = 5;
    float rotation = MathHelper.ToRadians(30); // 30 grados spread total

    for (int i = 0; i < numProjectiles; i++)
    {
        Vector2 perturbed = velocity.RotatedBy(MathHelper.Lerp(
            -rotation / 2, rotation / 2, i / (float)(numProjectiles - 1)));
        Projectile.NewProjectile(source, position, perturbed,
            type, damage, knockback, player.whoAmI);
    }
    return false;
}
```

20.3 Accesorio que da vida y defensa

```
public override void UpdateAccessory(Player player, bool hideVisual)
{
    player.statLifeMax2 += 20;           // +20 vida máxima
    player.statDefense += 4;             // +4 defensa
    player.lifeRegen += 2;               // +2 regen de vida por segundo
    player.endurance += 0.05f;          // +5% reducción de daño
}
```

20.4 ModSystem con hooks globales

```
using Terraria.ModLoader;

namespace MiMod.Content.Systems
{
    public class MiModSystem : ModSystem
    {
        public override void OnModLoad()
        {
            // Inicialización global
        }

        public override void PostUpdateWorld()
        {
            // Cada frame después de updateWorld
        }

        public override void PostSetupContent()
        {
            // Después de que todos los mods carguen - útil para cross-mod
            if (ModLoader.TryGetMod("CalamityMod", out Mod calamity))
            {
                // Setup cross-mod content
            }
        }
    }
}
```


20.5 GlobalItem que modifica todos los items

```
public class MiGlobalItem : GlobalItem
{
    public override void ModifyItemLoot(Item item, ItemLoot itemLoot)
    {
        // Añadir drops custom a jefes vanilla
        if (item.type == ItemID.KingSlimeBossBag)
        {
            itemLoot.Add(ModContent.ItemType<MiItemCustom>(), 0.1f); // 10%
chance
        }
    }

    public override void UpdateAccessory(Item item, Player player, bool
hideVisual)
    {
        // Modificar comportamiento de accesorios vanilla
        if (item.type == ItemID.CobaltShield)
        {
            player.moveSpeed += 0.1f; // Cobalt Shield da +10% movespeed
        }
    }
}
```

20.6 ModPlayer con estado custom

```
public class MiModPlayer : ModPlayer
{
    public int comboCounter; // Counter para combo system
    public bool hasSpecialBuff;

    public override void ResetEffects()
    {
        // Resetear cada frame (antes de aplicar accesorios)
        hasSpecialBuff = false;
    }

    public override void OnHitNPCWithItem(Item item, NPC target,
int damage, float knockback, bool crit)
    {
        if (hasSpecialBuff && item.DamageType == DamageClass.Melee)
        {
            comboCounter++;
            if (comboCounter >= 5)
            {
                // Trigger combo effect
                Projectile.NewProjectile(Player.GetSource_OnHit(target),
target.Center, Vector2.Zero,
ModContent.ProjectileType<ComboBurstProjectile>(),
damage * 2, 0f, Player.whoAmI);
                comboCounter = 0;
            }
        }
    }
}
```

21. Patrones de Generación para IA

Esta sección está específicamente orientada a un modelo de IA que use este documento como contexto para generar código de mods de Terraria. Cada regla es una instrucción operativa.

21.1 Reglas obligatorias al generar código

- TODO ModItem DEBE heredar de Terraria.ModLoader.ModItem
- TODO ModProjectile DEBE heredar de Terraria.ModLoader.ModProjectile
- TODO ModItem DEBE tener SetStaticDefaults() y SetDefaults() override
- TODO SetDefaults() DEBE asignar Item.DamageType
- TODO SetDefaults() DEBE asignar Item.width y Item.height
- TODO ModItem que sea arma DEBE tener Item.useStyle asignado
- TODO item que use mana DEBE tener Item.mana > 0
- TODO item ranged DEBE tener Item.useAmmo asignado
- TODO ModProjectile que sea minion DEBE tener Projectile.minion = true
- TODO receta DEBE llamar .Register() al final
- TODO ModPlayer DEBE overridear ResetEffects() para resetear estado
- En multiplayer, usar Projectile.ai[] para estado, NO campos estáticos

21.2 Plantilla de prompt sugerida

Cuando un usuario pida crear un arma, un buen prompt incluye: clase de daño, nivel aproximado de daño, mecánica especial (si la hay), tier de progresión (pre-hardmode, hardmode, post-Moon Lord), y sprite si es relevante. Ejemplo:

```
Prompt: "Crea un ModItem para tModLoader 1.4.4 que sea un arma mágica de tier hardmode con 45 de daño, maná 8, que dispare 3 proyectiles en abanico y aplique OnFire al impactar. La receta requiere 15 Souls of Light y un Spell Tome en un Bookcase."
```

21.3 Checklist de validación post-generación

Antes de entregar código generado, verificar mentalmente:

- ¿El namespace coincide con la estructura de carpetas?
- ¿La clase es public y no abstract?
- ¿SetStaticDefaults tiene DisplayName.SetDefault?
- ¿SetDefaults asigna width, height, damage, DamageType, useStyle, useTime, useAnimation?
- ¿El sprite path coincide con namespace.classname?
- ¿Si es summon, hay buffType configurado?
- ¿Si es ranged, useAmmo está asignado?

- ¿AddRecipes() llama .Register()?
- ¿Los hooks overrideados tienen la firma correcta (ref params, return types)?
- ¿No hay variables estáticas para estado de instancia?

21.4 Patrones anti-patrones que la IA debe evitar

- NO usar `item.melee = true` (es 1.3 legacy, usar `Item.DamageType = DamageClass.Melee`)
- NO usar `ModRecipe` (deprecado en 1.4, usar `Recipe.Create`)
- NO usar `Main.NewText` para logs (usar `Mod.Logger.Info`)
- NO usar `NPC.NewNPC` sin `SourceEntity` (causa problemas de sincronización)
- NO acceder a `Main.player[0]` directamente (usar `Projectile.owner` y `Main.player[Projectile.owner]`)
- NO usar `Texture = "MiMod/Items/..."` hardcoded (usar la convención automática o `($"{Name}")`)
- NO overridear `UseItem()` para spawnear proyectiles (usar `Shoot()`)

22. Recursos y Referencias Adicionales

22.1 Documentación oficial

Recurso	URL	Para qué sirve
tModLoader API docs	docs.tmodloader.net	Referencia completa de clases y hooks
tModLoader GitHub	github.com/tModLoader/tModLoader	Source code, issues, wiki
tModLoader Wiki	github.com/tModLoader/tModLoader/wiki	Tutoriales oficiales
Terraria Wiki	terraria.wiki.gg	Vanilla ItemID, ProjectileID, TileID, BuffID
Calamity Wiki	calamitymod.wiki.gg	Wiki del mod más grande
Thorium Wiki	thoriummod.wiki.gg	Wiki de Thorium

22.2 Comunidad

Recurso	URL	Para qué sirve
Foro Terraria (mods)	forums.terraria.org	Tutoriales, discusión, soporte
Reddit r/Terraria	reddit.com/r/Terraria	Noticias, ayuda general
Reddit r/terrariamods	reddit.com/r/terrariamods	Específico mods
Reddit r/CalamityMod	reddit.com/r/CalamityMod	Comunidad Calamity
Discord tModLoader	discord.gg/tmodloader	Soporte en tiempo real
Discord Calamity	discord.gg/calamity	Soporte Calamity

22.3 Herramientas y utilidades

Herramienta	URL	Para qué sirve
Steam Workshop tModLoader	steamcommunity.com/app/1281930	Publicar y descargar mods
tModLoader Mod Browser Mirror	mirror.sgkoi.dev	Mirror legacy del Mod Browser
Recipe Browser mod	Workshop ID 2498210918	Buscar recetas in-game
Cheat Sheet mod	Workshop ID 2825322950	Testing utilities
HEROs Mod	Workshop ID 2592348982	Spawn de items, NPC spawner
Structure Helper	Workshop ID 2790924965	Generar estructuras custom
Recipe Editor	Workshop ID 2620755310	Editar recetas in-game

22.4 Tutoriales recomendados

Tutorial	URL	Nivel
Modding Tutorial 1: Basics	forums.terraria.org/threads/118751	Principiante
Modding Tutorial 2: Basic Item & Sword	forums.terraria.org/threads/120094	Principiante
Modding Tutorial 3: Some More Weapons	forums.terraria.org/threads/135285	Intermedio
VS Templates for Items & Projectiles	gist.github.com/its-truce/a42026b4465f5b4f23cfc98653dc1c43	Intermedio
Projectile Guide	forums.terraria.org/threads/40062	Intermedio
Guide: Better VS Build	forums.terraria.org/threads/80009	Avanzado
Cross Mod Content wiki	github.com/tModLoader/tModLoader/wiki/Expert-Cross-Mod-Content	Avanzado

23. Conclusión y Recomendaciones

Este documento ha cubierto el ecosistema completo de modding de armas en Terraria con tModLoader 1.4.4: desde la instalación del entorno de desarrollo hasta patrones avanzados de cross-mod compatibility. La sección de catálogo recopila 12 repositorios de GitHub relevantes, cada uno con un enfoque distinto que sirve como referencia para diferentes necesidades. Las secciones de API (ModItem, ModProjectile, DamageClass) detallan los hooks exactos que un modder necesita conocer, con snippets de código funcionales listos para adaptar.

Para una IA que use este documento como contexto, las recomendaciones operativas son las siguientes. Primero, ante cualquier petición de generación de código de arma, verificar que se cumplen las reglas obligatorias del capítulo 21 (DamageType asignado, SetStaticDefaults presente, sprite path correcto, etc.). Segundo, para peticiones de análisis o comparación, usar el catálogo del capítulo 15 como referencia canónica. Tercero, para peticiones de extensión de mods existentes, consultar el capítulo 19 sobre cross-mod compatibility y el patrón de addon mods. Cuarto, mantener siempre presente el contexto de multiplayer: las variables estáticas están prohibidas para estado de instancia.

El ecosistema tModLoader es activo y en evolución. La versión 1.4.4 es la estable actual, pero la comunidad ya está experimentando con versiones preview y con la migración a .NET 8. Para estar al día, recomendamos suscribirse al Discord oficial de tModLoader, monitorizar los releases del repo GitHub principal, y revisar el foro forums.terraria.org para tutoriales nuevos. Los mods más grandes (Calamity, Thorium) son la mejor fuente de aprendizaje continuo: su código (cuando es público) muestra patrones avanzados que ningún tutorial cubre.

23.1 Próximos pasos sugeridos

- Si eres IA generando código: empezar con armas simples (espada básica del capítulo 5) antes de intentar mecánicas complejas

- Si eres desarrollador humano: clonar ExampleMod, compilarlo, y modificar un item existente como primer ejercicio
- Para mods serios: estudiar el source de CalamityModPublic para ver cómo se estructuran proyectos grandes
- Para cross-mod: revisar Calamity Entropy como ejemplo de addon bien estructurado
- Para performance: perfilar con dotnet-counters en servidor dedicado para identificar cuellos de botella
- Para comunidad: unirse al Discord de tModLoader y preguntar en #mod-help con logs adjuntos

23.2 Checklist final para cualquier mod de armas

- ¿Cada ModItem tiene SetStaticDefaults() + SetDefaults() override?
- ¿Item.DamageType está asignado en TODOS los SetDefaults()?
- ¿El sprite path coincide con namespace.classname?
- ¿AddRecipes() llama .Register() al final de cada receta?
- ¿Los hooks overrideados tienen la firma correcta?
- ¿No hay variables estáticas para estado de instancia?
- ¿build.txt tiene los modReferences/weakReferences correctos?
- ¿description.txt tiene una descripción útil para el menú in-game?
- ¿icon.png es 256x256 px y tiene transparencia?
- ¿Se ha testado en multiplayer además de singleplayer?